DEVELOPER-FACING TECHNICAL SPECIFICATION

RSCI FYP Portal

Software Design Document

Version 1.0

SYSTEM
Final Year Project Management Portal

INSTITUTION
Riphah School of Computing & Innovation

DATE
14 October 2026

STATUS
Baseline for technical review

Companion to the RSCI FYP Portal Software Requirements Specification v1.0

Architecture, components, data, security, workflows, deployment, and operations

FRONT MATTER

Document control

> **Document authority.** This SDD defines the recommended implementation design. The companion SRS defines required behavior. If they conflict, the approved SRS takes precedence and this SDD must be revised.

Document control

Field	Value
Document ID	RSCI-FYP-SDD-1.0
Title	RSCI FYP Portal - Software Design Document
Version / date	1.0 / 14 October 2026
Audience	Technology lead, developers, QA, DevOps, security reviewers
Classification	Internal academic system design
Approval required	Technology lead and designated RSCI process owners

Revision history

Version	Date	Change
1.0	14 Oct 2026	Initial complete technical baseline

How to use this document

- **Build teams:** implement module boundaries, contracts, state machines, and security controls in Sections 4-9.
- **QA teams:** derive integration and end-to-end tests from the transition tables and workflow sequences.
- **Process owners:** resolve items marked configurable or TBD before production rollout.
- **Technology lead:** ratify the proposed stack and Architecture Decision Records (ADRs).

> **Policy boundary.** Evaluation weights, stage names, deadlines, file limits, group-size limits, allocation caps, retention periods, and approval chains are not fixed by the supplied framework. Store them as versioned configuration until formally approved.

FRONT MATTER

Table of contents

Contents - core design

Section	Topic	Page
1	Introduction	5
2	System overview and architectural drivers	6
3	Recommended technology stack	7
4.1	Logical view	8
4.2	Deployment view	9
4.3	Data-flow views	10
5.1-5.2	M1 Authentication; M2 RBAC	11
5.3-5.4	M3 Sessions; M4 project catalogue	12
5.5-5.6	M5 groups; M6 proposal workflow	13
5.7-5.8	M7 allocation; M8 meetings and effort	14
5.9-5.10	M9 milestones; M10 repository	15
5.11-5.12	M11 AI compliance; M12 evaluation	16
5.12.1	Evaluation state model	17
5.13-5.15	M13 notifications; M14 reporting; M15 audit	18

Notation

Mark	Meaning
M#	Functional module identifier
SVC	Backend application service
POL	Configurable academic policy
ADR	Architecture decision record
TBD	Decision required before affected release

FRONT MATTER

Table of contents (continued)

Contents - workflows and controls

Section	Topic	Page
6.1	Proposal submission and two-stage approval	19
6.2	Deliverable submission and review loop	20
6.3	Evaluation scheduling through publication	21
6.4	Meeting, minutes, sign-off, effort rollup	22
7.1	Data design: academic and project entities	23
7.2	Data design: execution, evaluation, evidence	24
8.1	Authentication, authorization, data protection	25
8.2	OWASP-aligned controls and audit	26
9.1	Errors, logging, observability, API conventions	27
9.2	Caching, jobs, files, consistency	28
10	Performance and scalability	29
Appendix A	Architecture Decision Records	30
Appendix B	Open design issues / TBD	31
Appendix C	Implementation handoff and glossary	32

Structured diagrams. Each figure uses named nodes and arrows, followed by a prose description. Tables define responsibilities, transitions, and data ownership so the design remains accessible without relying on color or graphics alone.

SECTION 1

1. Introduction

1.1 Purpose

This Software Design Document (SDD) translates the approved requirements of the RSCI FYP Portal into an implementable technical design. It defines architecture, components, service boundaries, state machines, data ownership, security controls, deployment, operational behavior, and decision points for development and verification.

1.2 Scope

The portal supports the complete Final Year Project (FYP) lifecycle across academic sessions: user and role administration; multi-domain project policies; student groups; proposals; supervisor allocation; meetings and effort; milestones and versioned evidence; artificial intelligence (AI) declarations; rubric-based evaluation; notifications; analytics; and audit history.

1.3 References

Reference	Use in this design
RSCI FYP Portal SRS v1.0	Functional and non-functional behavioral baseline
Riphah School of Computing & Innovation, FYP Framework v1.0	Ten project types, deliverables, AI-usage rules, learning objectives, 9 hours/week and 300-hour expectation
OWASP Application Security Verification Standard / Top 10	Security-control design reference; edition selected by security lead
Institutional policies	Pending: identity, retention, assessment, moderation, and records rules

1.4 Overview and boundaries

- The system is a responsive web application; native mobile apps are outside v1.0 unless separately approved.
- The portal records and controls academic workflow; it does not replace an official learning management system, student information system, or finance system without an approved integration.
- The design is technology-specific only where marked **Recommended**; final stack selection belongs to the technology lead.

SECTION 2

2. System overview and architectural drivers

The portal is workflow-heavy and evidence-centric. The design therefore favors explicit state transitions, immutable versions, contextual authorization, and configuration snapshots over ad hoc form updates.

2.1 Architectural drivers

Driver	Design response
Academic-session isolation	Every operational record carries session scope; queries and permissions require it.
RBAC everywhere	Endpoint guard plus resource-level policy check; deny by default.
Approval workflows	State machines, transition commands, optimistic locking, history events.
Document versioning	Immutable object versions with hashes, metadata, and active-version pointer.
Configurable rubrics	Versioned templates; evaluation snapshots preserve the exact criteria used.
Auditability	Append-only audit events for sensitive reads, writes, decisions, and exports.
Multi-domain FYPs	Project-type policy controls deliverables, AI rules, and evaluation overlays.
Deadline pressure	Queues, idempotency, bulk operations, autosave, and graceful retries.

2.2 Design principles

- **Single source of truth:** PostgreSQL owns transactional metadata; object storage owns file bytes.
- **Configuration, not code:** academic stages, deadlines, scoring bands, and limits are versioned data.
- **History over overwrite:** revisions create versions; decisions create events.
- **Least privilege:** role permission is necessary but not sufficient; session, group, project, and panel scope are checked.
- **Modular first:** one deployable application may contain strongly separated modules and schemas.

> **Multi-tenancy meaning.** In v1.0, tenant isolation means academic-session and organizational-scope separation inside one institution. It does not imply a public multi-institution SaaS product.

2.3 Role contexts

Role	Primary design scope
Super Admin	Platform configuration and controlled administration
FYP Coordinator	Session workflow, assignment, scheduling, and publication control
HOD	Program-level decisions and oversight
Supervisor	Assigned projects, meetings, reviews, and progress
Co-Supervisor	Delegated assigned-project collaboration
Student Leader/Member	Own group, project evidence, declarations, and feedback
Internal Evaluator	Assigned panel and criterion scoring
External Examiner	Assigned examination and restricted evidence
Industry Partner	Linked project input, acceptance, or feedback as configured

SECTION 3

3. Recommended technology stack

> **Recommendation - decision required.** The stack below is a proposed baseline. The technology lead must select the backend option and record the decision before implementation begins.

Layer	Recommended technology	Rationale
Frontend	React + TypeScript, Next.js	Typed component model, SSR/CSR flexibility, mature forms and testing.
Backend	Node.js + NestJS *or* Django REST Framework	Both support modular APIs, validation, authorization, jobs, and OpenAPI.
Database	PostgreSQL 15+	Transactions, relational integrity, JSONB, indexing, reporting.
Cache / session	Redis	Rate limits, short-lived cache, job coordination, token denylist if used.
Documents	S3-compatible object storage	Durable versioned blobs, signed access, lifecycle policies.
Authentication	Short-lived JWT access + rotated refresh tokens	Stateless API access with controlled renewal and revocation.
Async	Queue and worker processes	Notifications, virus scans, imports, exports, scheduled reminders.
Edge / runtime	Nginx, Docker, CI/CD pipeline	TLS termination, routing, repeatable deployment, automated quality gates.

3.1 Backend alternatives

Option	Strength	Trade-off
NestJS	One TypeScript language; strong module/decorator patterns	Runtime typing requires disciplined validation
Django REST Framework	Mature ORM/admin/security ecosystem	Separate frontend/backend language stack

3.2 Required pipeline gates

- Lint, type check, unit tests, migration check, dependency scan, secret scan, SAST, API contract validation, container scan, integration tests, and signed deployment artifact.
- Production deployment requires reviewed migration, backup checkpoint, health checks, and rollback plan.

SECTION 4 - ARCHITECTURAL VIEWS

4.1 Logical view



Figure 1 - logical architecture. Requests move downward through validation and use-case orchestration; domain rules remain independent of HTTP and storage. Domain events move upward to notification, reporting, and audit adapters.

4.1 Layer responsibilities

Layer	Responsibilities	Must not do
Presentation	Routes, forms, accessibility, client state, optimistic UI, role-aware navigation	Decide authorization or scoring rules
API / Application	DTO validation, authentication context, commands, transactions, idempotency, orchestration	Embed UI-specific behavior
Domain	Entities, policies, state machines, scoring, invariants, domain events	Call web frameworks or object storage directly
Persistence / Adapters	Repositories, SQL, object storage, queue, email/SMS adapters	Bypass domain validation

4.1.1 Module boundary rule

A module exposes application interfaces and events, not its internal tables. Cross-module writes occur through a service command in one transaction where possible. Read models may join approved views for dashboards but do not become alternate write paths.

4.1.2 Shared kernel

- Identifiers, actor context, session scope, money-free date/time value objects, pagination, audit metadata, error types, event envelope, and policy-evaluation result.
- The shared kernel must remain small; academic logic belongs to the owning module.

SECTION 4 - ARCHITECTURAL VIEWS

4.2 Deployment view

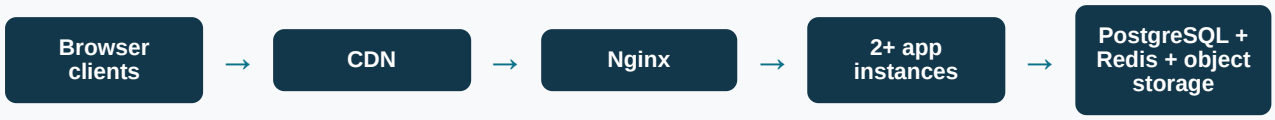


Figure 2 - deployment topology. Static assets terminate at the CDN. Dynamic HTTPS requests pass through Nginx to stateless application instances. Workers consume durable jobs and use the same database, cache, and object storage through private network paths.

4.2 Deployment nodes

Node	Purpose	Availability / security notes
CDN	Cache hashed frontend assets	No personalized API response caching
Nginx	TLS, routing, request limits, security headers	Only public ingress; preserve correlation ID
App instances	REST API and server rendering	At least two in production; stateless
Worker nodes	Mail, reminders, scans, imports, exports	Separate queues and concurrency limits
PostgreSQL	System of record	Primary plus optional read replica; PITR backups
Redis	Cache, rate limits, queue support	Private; persistence depends on queue design
Object storage	Versioned files and exports	Private buckets; short-lived signed access

> **Network boundary.** Database, Redis, worker administration, and object storage management endpoints remain on private networks. Public clients never receive storage credentials or direct database connectivity.

4.2.1 Environments

Use isolated development, test, staging, and production environments with separate databases, buckets, secrets, and identity clients. Production data must not be copied to lower environments without approved anonymization.

SECTION 4 - ARCHITECTURAL VIEWS

4.3 Data-flow descriptions

4.3 Data-flow catalogue

Flow	Write path	Events / outputs
Proposal approval	Draft -> submit -> validate -> coordinator review -> HOD decision -> approved snapshot	ProposalSubmitted, ReviewRecorded, ProposalApproved/Returned/Rejected
Deliverable review	Upload -> scan -> version -> AI declaration -> submit -> supervisor review -> accept/changes	SubmissionCreated, ScanCompleted, ReviewRequested, SubmissionAccepted
Evaluation	Schedule -> panel assignment -> score drafts -> lock -> aggregate -> moderation -> publish	EvaluationLocked, ModerationAdjusted, ResultPublished
Meeting sign-off	Schedule -> attendance -> minutes -> participant acknowledgement -> supervisor sign-off -> effort rollup	MeetingHeld, MinutesSigned, EffortUpdated

4.3.1 Transaction and event pattern

1. API validates the command, actor context, session scope, and idempotency key.
2. Application service loads aggregates and policy snapshots inside a database transaction.
3. Domain method checks state transition and emits domain events.
4. Repository persists changed records, transition history, audit event, and outbox event atomically.
5. Worker publishes notifications and read-model updates from the outbox.
6. Consumers store event IDs so retries remain idempotent.

4.3.2 Consistency rules

- Proposal state, review record, and transition history commit atomically.
- File bytes may upload before metadata commit, but uncommitted objects stay quarantined and are garbage-collected.
- Notification delivery is eventually consistent; academic decisions are not rolled back because an email fails.
- Dashboard counts may lag by a documented short interval; authoritative detail screens read transactional data.

> **Failure behavior.** If an external channel or export fails, preserve the academic transaction, mark the job failed, retry under policy, and expose status to authorized administrators. Never duplicate scores, submissions, or notifications on retry.

SECTION 5 - COMPONENT DESIGN

5.1-5.2 Identity and authorization components

5.1 M1 - Authentication & User Management

Service / class	Responsibilities
AuthService	Login, token issue/rotation/revocation, lockout, password reset initiation, session list.
UserService	Create/import users, status, profile, verified identifiers, institutional mapping.
RefreshTokenStore	Store only hashed refresh tokens; family rotation; reuse detection; device metadata.
AuthenticationGuard	Validate issuer, audience, signature, expiry, token version, and user status.

Core interactions

- User import creates inactive or invited accounts; activation status is separate from role assignment.
- Login returns a short-lived access token and a protected refresh mechanism. Browser storage choice is an ADR; prefer secure, HTTP-only, SameSite cookies for refresh tokens.
- Password reset invalidates prior refresh families after successful reset.

5.2 M2 - Role-Based Access Control (RBAC)

Service / class	Responsibilities
RoleService	Manage role definitions and assignments within organizational/session scope.
PermissionService	Resolve permissions from role, scope, delegation, and record context.
RBACGuard	Reject endpoints lacking required permission before service execution.
ResourcePolicy	Enforce group, project, supervisor, panel, evaluator, and session ownership checks.

> **Enforcement invariant.** Hiding a button is never authorization. Every command and sensitive query must pass both endpoint permission and resource-policy checks. Super Admin access is logged and remains constrained by explicit permissions.

SECTION 5 - COMPONENT DESIGN

5.3-5.4 Sessions and multi-domain catalogue

5.3 M3 - Academic Session & Batch Management

Service / class	Responsibilities
SessionService	Create, activate, close, archive sessions; enforce one configured active session per scope.
BatchService	Manage programs, cohorts, sections, terms, and student enrollment mappings.
CalendarService	Own milestones, blackout dates, stage windows, and timezone-aware deadlines.
PolicySnapshotService	Freeze approved configuration versions used by projects and evaluations.

5.4 M4 - Multi-Domain Project Catalogue

Project type	Programs in framework
Product Development Project	CS, SE, IT
Entrepreneurial Project	CS, SE, IT
Continuation Project (Existing product based Project)	CS, SE, IT, CY, DS, AI, CGV
Research Project	CS, AI, CY, DS
IoT & Embedded Systems Project	CS, SE, IT
Case Study Project	IT, CY, DS
Consulting Project (AI/Data Analytics)	AI, DS
Consulting Project (Cybersecurity)	CY, IT
Consulting Project (Cloud & DevOps)	IT, CS, SE
Consulting Project (Digital Transformation)	IT, CS, SE

ProjectTypeCatalogService versions descriptions, program eligibility, deliverable templates, AI policy, outcome tags, and rubric overlays. **PolicyEngine** resolves the effective configuration for a project using its type and session snapshot. Historical projects never silently inherit later catalogue edits.

SECTION 5 - COMPONENT DESIGN

5.5-5.6 Groups and proposal workflow

5.5 M5 - Group Formation & Registration

Service / class	Responsibilities
GroupService	Create group, invite/remove members, transfer leader, lock roster, archive group.
EligibilityPolicy	Check enrollment, program, session, duplicate membership, and configurable size rules.
InvitationService	Issue expiring invitation; record accept/decline; prevent replay.

A group belongs to one academic session and project type. Student Leader/Member is a contextual role on `group_membership`, not a global user role. Roster edits after lock require authorized override with reason and audit event.

5.6 M6 - Proposal Submission & Approval Workflow

ProposalWorkflowService owns the aggregate, delegates validation to **ProposalPolicy**, writes reviews through **ProposalReviewService**, and emits outbox events. The two-stage route is configurable; the baseline sequence is coordinator review then HOD decision.

Current	Command / actor	Next	Required effect
DRAFT	submit / Student Leader	SUBMITTED	Freeze submitted version; validate roster and required fields
SUBMITTED	start review / Coordinator	UNDER_COORDINATOR_REVIEW	Assign reviewer; record start
UNDER_COORDINATOR_REVIEW	return / Coordinator	CHANGES_REQUESTED	Reason required; notify group
CHANGES_REQUESTED	resubmit / Student Leader	SUBMITTED	Create new immutable version
UNDER_COORDINATOR_REVIEW	recommend / Coordinator	UNDER_HOD_REVIEW	Record recommendation
UNDER_HOD_REVIEW	approve / HOD	APPROVED	Create approved snapshot; enable allocation/milestones
UNDER_HOD_REVIEW	return / HOD	CHANGES_REQUESTED	Reason required
UNDER_HOD_REVIEW	reject / HOD	REJECTED	Reason required; terminal unless reopened

> **Concurrency.** Every transition includes the current version number. A stale command returns HTTP 409 and the latest state; it never overwrites a newer review.

SECTION 5 - COMPONENT DESIGN

5.7-5.8 Allocation, meetings, and effort

5.7 M7 - Supervisor Allocation

Service / class	Responsibilities
AllocationService	Create candidate assignments, confirm/decline, co-supervisor links, reassign with history.
CapacityPolicy	Compute load against configurable caps and approved exceptions.
MatchingService	Rank declared expertise, project type, preferences, conflicts, and availability.
ConflictService	Record disclosure and recusal; block prohibited assignments.

Matching output is advisory. An authorized coordinator confirms allocations. Reassignment preserves the original assignment, reason, effective dates, and open review responsibilities.

5.8 M8 - Meeting Catalogue & Logbook

Service / class	Responsibilities
MeetingService	Schedule/cancel meetings, attendance, agenda, minutes, decisions, actions, sign-off.
EffortTrackerService	Record category-level hours, evidence, verification, weekly and cumulative rollups.
ActionItemService	Assign owners/dates; close or carry actions; notify overdue owners.
MeetingPolicy	Validate participant scope and required sign-off.

Current	Action	Next
SCHEDULED	cancel	CANCELLED
SCHEDULED	record attendance/minutes	MINUTES_DRAFT
MINUTES_DRAFT	submit for sign-off	AWAITING_SIGNOFF
AWAITING_SIGNOFF	request correction	MINUTES_DRAFT
AWAITING_SIGNOFF	supervisor signs	SIGNED
SIGNED	authorized amendment	AMENDED

> **Effort rule.** The framework states 9 hours per week and 300 hours for a complete project. The portal reports progress and exceptions; whether it blocks submission is a configurable policy decision, not assumed here.

SECTION 5 - COMPONENT DESIGN

5.9-5.10 Milestones and repository

5.9 M9 - Milestone & Deliverable Tracking

Service / class	Responsibilities
MilestoneService	Instantiate session/type templates; dates, dependencies, status, extensions.
SubmissionService	Create submission attempts, validate window, attach versions, submit and withdraw.
ReviewService	Structured comments, decision, required changes, acceptance, reviewer assignment.
DeadlinePolicy	Resolve timezone, grace period, approved extension, and late label.

5.10 M10 - Document Repository & Version Control

Service / class	Responsibilities
DocumentService	Metadata, logical document, immutable versions, access checks, active version.
UploadService	Quarantine, size/type checks, checksum, malware scan, object promotion.
DownloadService	Authorize and issue short-lived signed URL or streamed response.
RetentionService	Retention/lifecycle jobs; legal hold and deletion authorization.

Submission state machine

Current	Command	Next	Notes
DRAFT	submit	PROCESSING	Idempotency key; files must exist
PROCESSING	scan passed	SUBMITTED	Record submitted_at and policy snapshot
PROCESSING	scan failed	QUARANTINED	No reviewer access; uploader notified
SUBMITTED	start review	UNDER_REVIEW	Reviewer assignment required
UNDER_REVIEW	request changes	CHANGES_REQUESTED	Structured feedback required
CHANGES_REQUESTED	new version	SUBMITTED	Prior version remains immutable
UNDER_REVIEW	accept	ACCEPTED	Decision and actor recorded
SUBMITTED	withdraw	WITHDRAWN	Only while policy allows

> **Immutability.** Replace means create a new version. Never overwrite object bytes or edit the submitted version in place.

SECTION 5 - COMPONENT DESIGN

5.11-5.12 AI compliance and evaluation

5.11 M11 - AI-Usage Declaration & Compliance

Service / class	Responsibilities
AIDeclarationService	Capture tool, purpose, scope, affected artifacts, dates, student attestation, supervisor review.
AIPolicyEngine	Evaluate declaration against project-type policy snapshot and deliverable context.
ComplianceCaseService	Open exception, request evidence, record disposition, preserve restricted access.

The project-type catalogue carries the framework rules. Examples include extensive AI use for Entrepreneurial Projects, API-based AI application where allowed, and documentation restrictions for multiple types. The engine stores the matched rule ID and version with every result; it does not infer misconduct from text similarity or automated detection alone.

5.12 M12 - Evaluation & Rubric Management

Service / class	Responsibilities
RubricService	Version templates, criteria, performance levels, weights, applicability, approval.
EvaluationScheduleService	Stages, rooms/links, panels, conflicts, attendance, rescheduling.
EvaluationService	Draft scores, comments, validation, lock, aggregate, release eligibility.
ModerationService	Outlier flags, review queue, adjustments with reason and approvals.
ResultPublicationService	Preview, approve, publish, retract under exceptional authority.

Scoring invariants

- Use decimal-safe arithmetic; store raw criterion scores and derived totals separately.
- Snapshot rubric version, stage configuration, panel membership, and aggregation rule at evaluation creation.
- An evaluator cannot score before assignment or after lock except through audited reopen.
- Moderation never overwrites original scores; it adds an adjustment record and rationale.

SECTION 5 - COMPONENT DESIGN

5.12.1 Evaluation and rubric states

5.12.1 Evaluation state machine

Current	Command / actor	Next	Guards and effects
PLANNED	schedule / Coordinator	SCHEDULED	Stage, project, panel, rubric snapshot present
SCHEDULED	open / Coordinator/system	OPEN	Window started; conflicts resolved
OPEN	save draft / Evaluator	OPEN	Criterion validation; autosave with version
OPEN	lock score / Evaluator	PARTIALLY_LOCKED	Evaluator attests completion
PARTIALLY_LOCKED	last panel lock	LOCKED	All required panel members locked
LOCKED	aggregate / system	AGGREGATED	Apply snapshotted rule; decimal rounding once
AGGREGATED	flag / system/Coordinator	MODERATION_REQUIRED	Threshold is configurable
AGGREGATED	approve / Coordinator	READY_TO_PUBLISH	No unresolved flags
MODERATION_REQUIRED	resolve / Moderator	READY_TO_PUBLISH	Reason, evidence, authority recorded
READY_TO_PUBLISH	publish / authorized role	PUBLISHED	Release snapshot; notify permitted audience
PUBLISHED	retract / exceptional authority	RETRACTED	Reason required; previous release retained

Aggregation contract

1. Validate all required criteria and evaluator locks.
2. Convert each criterion score to its configured normalized contribution.
3. Apply stage and evaluator aggregation rules from the snapshot.
4. Calculate raw total at high precision; round only at the configured display/final boundary.
5. Store component breakdown, formula version, total, flags, and calculation timestamp.

Rubric version lifecycle



Figure 3 - rubric lifecycle. Only an approved version may become active. Existing evaluations retain their snapshot after a newer version is activated. Retired versions remain readable for audit.

> **No invented weights.** The supplied framework does not define evaluation stages, marks, or weightages. All such values must be entered and approved by authorized academic owners before use.

SECTION 5 - COMPONENT DESIGN

5.13-5.15 Communication, reporting, and audit

5.13 M13 - Notifications & Communication

Service / class	Responsibilities
NotificationService	Create in-app records from events; audience resolution; deduplication.
TemplateService	Versioned channel templates with safe variables and preview.
DeliveryWorker	Email/SMS/in-app dispatch, retry, dead-letter, provider status.
PreferenceService	Permitted user preferences; mandatory academic notices cannot be silenced.

5.14 M14 - Dashboards, Analytics & Reporting

Service / class	Responsibilities
DashboardQueryService	Role-scoped counts, deadlines, workload, progress, exceptions.
ReportingService	Parameterized reports, authorization, asynchronous export, expiry.
ReadModelProjector	Update denormalized summaries from outbox events; rebuildable.
MetricDefinitionRegistry	Own formula, filters, source fields, and version for each metric.

5.15 M15 - Audit Trail

Service / class	Responsibilities
AuditService	Append actor, action, target, before/after summary, reason, IP/device metadata, correlation ID.
AuditQueryService	Authorized search/export with field redaction and access logging.
IntegrityJob	Sequence/hash checks where adopted; detect missing or altered events.
RetentionPolicy	Apply approved retention and legal holds; protect academic records.

> **Audit content.** Do not store passwords, tokens, full file contents, or unnecessary personal data in audit payloads. Record references, changed field names, protected summaries, and reason codes.

SECTION 6 - KEY WORKFLOWS

6.1 Proposal submission and two-stage approval

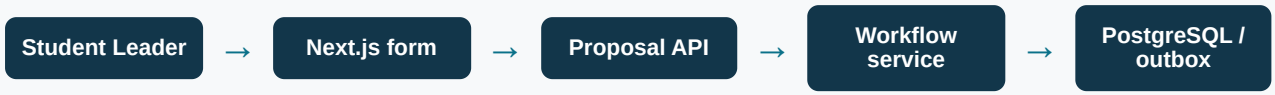


Figure 4 - proposal sequence. The coordinator and HOD later use the same command path; only the command, actor policy, and transition differ.

6.1 Numbered sequence

1. **Draft:** frontend loads the group, effective project-type policy, required fields, and current proposal version using an ETag.
2. **Submit:** Student Leader sends validated content, version, and idempotency key to `POST /v1/proposals/{id}/submit`.
3. **Authorize:** AuthenticationGuard validates identity; RBACGuard checks `proposal.submit`; ResourcePolicy confirms group leadership and session scope.
4. **Validate:** ProposalWorkflowService verifies roster, required evidence, project-type eligibility, deadline/extension, and current DRAFT or CHANGES_REQUESTED state.
5. **Commit:** service stores an immutable proposal version, transition event, audit record, and ProposalSubmitted outbox event in one transaction.
6. **Coordinator review:** assigned coordinator opens the snapshot, records comments, then returns it or recommends it to the HOD. Each command uses optimistic locking.
7. **HOD review:** authorized HOD approves, returns, or rejects. Approval creates the approved snapshot used by allocation and milestone modules.
8. **Notify:** worker consumes outbox events, creates in-app messages, dispatches permitted channels, and records delivery outcome.

Error and retry behavior

Condition	Response
Missing required field	422 with field-level errors
Stale version	409 with current state/version
Duplicate idempotency key	Return original result
Unauthorized group	404 or 403 per disclosure policy
Notification provider failure	Keep decision; retry delivery job

SECTION 6 - KEY WORKFLOWS

6.2 Deliverable submission and review loop

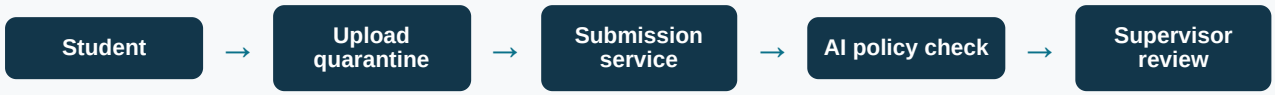


Figure 5 - deliverable loop. File promotion follows scan success. A changes request creates a new attempt/version rather than rewriting prior evidence.

6.2 Numbered sequence

1. **Prepare upload:** frontend requests an authorized upload intent for the project, milestone, logical document, declared media type, and size.
2. **Upload:** client sends bytes to a private quarantine location; storage key is server-generated and unguessable.
3. **Verify:** worker checks byte signature, configured size/type, checksum, and malware result; it promotes clean content and rejects unsafe files.
4. **Declare AI use:** student completes the required declaration fields. AIPolicyEngine evaluates them against the project-type and deliverable policy snapshot.
5. **Submit:** SubmissionService validates deadline, required files, declaration status, and attempt policy; then commits SUBMITTED state and audit/outbox events.
6. **Review:** supervisor records criterion/section comments, attachments if allowed, and ACCEPT or CHANGES_REQUESTED. The decision and feedback version are immutable.
7. **Revise:** student creates a new document version and submission attempt linked to the prior one. The reviewer can compare metadata and download authorized versions.
8. **Close:** acceptance updates milestone completion and publishes the permitted status to dashboards.

Data ownership

Record	Owner
File bytes	Object storage
Document/version metadata	M10 Repository
Submission attempt/status	M9 Milestones
AI declaration/result	M11 Compliance
Review and decision	M9 Review service
Notifications	M13 Notification module

SECTION 6 - KEY WORKFLOWS

6.3 Evaluation through publication



Figure 6 - evaluation sequence. A snapshotted rubric and panel drive scoring. Original scores remain preserved even when moderation adds an adjustment.

6.3 Numbered sequence

1. **Configure:** authorized owner activates an approved rubric version and stage configuration for the academic session and project type.
2. **Schedule:** coordinator selects projects, window, venue/link, and panel. ConflictService blocks duplicate or conflicted assignments according to approved rules.
3. **Snapshot:** EvaluationScheduleService stores rubric, aggregation rule, panel, and stage snapshots so later edits cannot alter the event.
4. **Score:** each evaluator saves criterion-level drafts using optimistic locking. The API validates bounds, mandatory comments, and assignment scope.
5. **Lock:** evaluator attests completion. Reopening requires authorized command, reason, and audit event.
6. **Aggregate:** after all required locks, EvaluationService calculates the breakdown with decimal-safe arithmetic and emits flags using configurable thresholds.
7. **Moderate:** designated moderator reviews evidence and records no-change or an additive adjustment. The original panel record remains intact.
8. **Approve:** coordinator reviews publication preview, unresolved flags, missing evaluations, and visibility rules.
9. **Publish:** ResultPublicationService creates a release snapshot, changes status to PUBLISHED, and notifies only authorized recipients.
10. **Correct:** exceptional post-publication correction uses retract-and-republish; no silent edit is permitted.

Transaction boundaries

Each evaluator lock is independent. Aggregation begins only when completion guards pass. Publication is a separate privileged transaction so a successful calculation cannot accidentally expose results.

SECTION 6 - KEY WORKFLOWS

6.4 Meeting sign-off and effort rollup

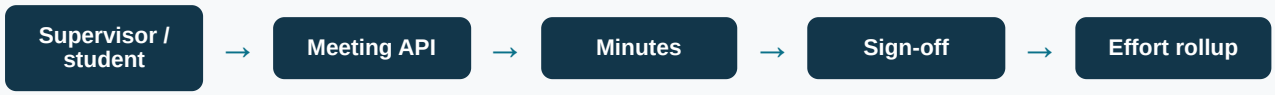


Figure 7 - meeting and effort sequence. A signed meeting is evidence. Effort lines remain attributable to people, categories, dates, and source records.

6.4 Numbered sequence

1. **Schedule:** authorized participant proposes time, mode, agenda, and participants; MeetingPolicy confirms project membership and session.
2. **Notify:** MeetingScheduled event creates invitations and reminders. External calendar integration is optional and must not become the system of record.
3. **Conduct:** recorder marks attendance and actual times, enters discussion, decisions, risks, and action items with owners/dates.
4. **Log effort:** members add date, hours, category, description, and evidence link. Values are decimal hours and constrained by configurable sanity limits.
5. **Submit minutes:** recorder sends the minutes snapshot to participants/supervisor. Further edits create a revision.
6. **Correct:** supervisor requests correction with reason; state returns to MINUTES_DRAFT without deleting prior snapshot.
7. **Sign:** supervisor signs the accepted version. Service records actor, timestamp, version hash, and transition event.
8. **Roll up:** EffortTrackerService recalculates weekly and cumulative verified/unverified totals and checks 9-hour and 300-hour indicators.
9. **Follow up:** actions appear on dashboards; overdue reminders are queued. Amendments after sign-off preserve the signed original and reason.

Rollup formula

`verified_total = SUM(effort_entry.hours WHERE status = VERIFIED AND not superseded)`. Weekly boundaries use the institution-approved timezone and week start. Never derive hours from meeting duration alone; effort entries are explicit and attributable.

SECTION 7 - DATA DESIGN

7.1 Data design summary - academic and project

> **Design boundary.** This section is a logical summary. Column types, keys, indexes, constraints, migration order, and physical naming belong in the companion Database Design document.

7.1 Academic, identity, and project entities

Entity	Purpose / key relationships
User	Identity profile; has role assignments and actor history.
Role	Named permission bundle; institutional or contextual.
Permission	Atomic action such as proposal.approve or result.publish.
RoleAssignment	User + role + organizational/session scope + validity.
AcademicSession	Top-level operational scope; owns calendar and policy snapshots.
Program	CS, SE, IT, CY, DS, AI, CGV catalogue entry.
Batch / Cohort	Program intake grouping; contains enrollments.
Enrollment	User as student in program, batch, session, and status.
ProjectTypeVersion	Versioned multi-domain policy and eligible programs.
DeliverableTemplate	Required evidence definition linked to type/session policy.
Group	Student team in one session and selected project type.
GroupMembership	Group + student + leader/member context + effective dates.
GroupInvitation	Expiring invite with issuer, recipient, and outcome.
Project	Approved work identity linked to group, type, and session.
Proposal	Logical proposal owned by group; has many versions.
ProposalVersion	Immutable content/evidence snapshot and submitter.
ProposalReview	Stage, reviewer, outcome, comments, and proposal version.
SupervisorAssignment	Project + supervisor/co-supervisor + effective history.
ConflictDisclosure	Actor/project relationship, status, resolution, evidence.

Relationship invariants

- A student has at most one active group membership per academic-session scope unless an approved exception exists.
- A project refers to one approved proposal version and one project-type version snapshot.
- Supervisor reassignment closes the prior assignment; it does not update it in place.
- User deactivation preserves authored records and displays a historical identity reference.

SECTION 7 - DATA DESIGN

7.2 Data design summary - execution and evidence

7.2 Execution, evidence, and evaluation entities

Entity	Purpose / key relationships
Meeting	Project meeting lifecycle, schedule, actuals, and signed revision.
MeetingParticipant	Attendance, role, acknowledgement, sign-off metadata.
ActionItem	Meeting-derived owner, due date, status, completion.
EffortEntry	Member/date/category/hours/evidence and verification.
Milestone	Project instance of a template, dates, status, extension.
Submission	Milestone attempt with lifecycle and policy snapshot.
Document	Logical artifact with owner and classification.
DocumentVersion	Immutable storage key, hash, scan status, metadata.
AIDeclaration	Declared tool/purpose/scope/artifacts and attestation.
ComplianceCase	Policy result, evidence, reviewer, disposition.
RubricVersion	Approved template snapshot and applicability.
RubricCriterion	Hierarchy, scale, weight, levels, validation.
EvaluationEvent	Project/stage/schedule/rubric/panel snapshot.
PanelAssignment	Evaluator role, conflict status, required flag.
CriterionScore	Evaluator + criterion + raw score + comment + version.
EvaluationResult	Calculated breakdown, formula version, total, status.
ModerationRecord	Flag, review, adjustment, reason, authority.
Notification	Audience, template, channel, state, dedupe key.
OutboxEvent	Durable domain event pending asynchronous handling.
AuditEvent	Append-only sensitive action and correlation metadata.

Database implementation rules

- Use UUID/ULID-style opaque identifiers; never expose sequential IDs if they reveal record volume.
- Add `created_at`, `created_by`, and version where concurrency matters; use timezone-aware UTC timestamps.
- Prefer status constraints/enums managed by migrations; preserve transition history in separate tables.
- Index session/status/deadline, foreign keys, assignment lookups, unread notifications, outbox pending state, and normalized search columns.
- Use database uniqueness and check constraints for invariants that survive concurrent requests.

SECTION 8 - SECURITY DESIGN

8.1 Authentication, authorization, and protection

8.1 Authentication flows

Flow	Design
Login	Rate-limited credential check; Argon2id preferred or bcrypt with reviewed cost; generic failure message; account status check.
Access token	Short lifetime; issuer, audience, subject, roles/version, issued/expiry IDs; asymmetric signing preferred.
Refresh	Opaque or JWT refresh token stored protected; server stores hash; rotate every use; revoke family on reuse.
Logout	Revoke refresh family; optionally denylist access token ID for high-risk sessions.
Password reset	Single-use, short-lived random token stored hashed; invalidate after use; audit without token value.
MFA	Required design extension for privileged roles; method and rollout are TBD.

8.2 Authorization enforcement

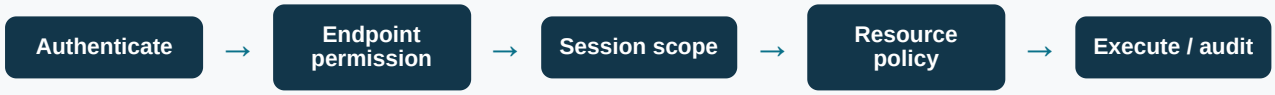


Figure 8 - authorization pipeline. A request reaches business logic only after identity, permission, session, and record relationship checks succeed.

- Controller declares required permission; RBACGuard resolves active assignments.
- Application service passes ActorContext to ResourcePolicy for group, project, evaluator, or supervisor scoping.
- Repository queries include authorized session/project filters; post-filtering alone is prohibited.
- Bulk actions authorize every target and return per-item errors or an all-or-nothing result as specified.

8.3 Data and file protection

- TLS 1.2+ or stronger approved profile in transit; managed encryption for database, backups, and object storage at rest.
- Private buckets, short-lived signed URLs, content-disposition controls, and authorization on every issue of a URL.
- Upload allowlist by byte signature, configurable limits, filename normalization, malware scan, quarantine, and archive-bomb defenses.
- Grade and moderation data are restricted before publication; exports inherit the same scope and expire.

SECTION 8 - SECURITY DESIGN

8.2 OWASP controls and security audit

8.4 OWASP-aligned threat controls

Risk area	Portal control
Broken access control	Deny-by-default guards, resource policy, scoped queries, IDOR tests, privileged-action audit.
Cryptographic failures	TLS, managed encryption, protected secrets, strong password hash, token rotation.
Injection	Parameterized ORM/SQL, schema validation, escaped templates, no shell composition.
Insecure design	Threat modeling for grades/files/workflows; state machines; approval separation.
Misconfiguration	Hardened images, least-privilege IAM, security headers, environment validation.
Vulnerable components	Lockfiles, dependency/container scans, patch SLA, SBOM.
Auth failures	Rate limits, lockout policy, MFA extension, session revocation, generic errors.
Integrity failures	Signed build artifacts, protected CI, checksums, immutable versions, reviewed migrations.
Logging failures	Structured security events, alert rules, correlation IDs, protected log store.
SSRF / unsafe fetch	No arbitrary server fetch; destination allowlist; block private metadata ranges.

8.5 Sensitive-action audit events

Action	Minimum audit facts
Role/permission change	Actor, subject, old/new scope, reason
Proposal/evaluation decision	Actor, target/version, transition, reason
Score lock/reopen/moderation	Evaluator/moderator, before/after summary, authority
Result publish/retract	Release snapshot, audience, reason
File access/export	Actor, classification, target set, outcome
Impersonation/support access	Initiator, subject, approved purpose, duration

> **Privacy.** Collect only data required for academic workflow. Retention, deletion, data-subject handling, and breach response must follow approved institutional and legal policy; values remain TBD in this design.

SECTION 9 - CROSS-CUTTING CONCERNS

9.1 Errors, API conventions, and observability

9.1 Standard API error envelope

All non-2xx responses use one machine-readable shape. Never expose stack traces, SQL, storage keys, or secrets to clients.

```
{
  "error": {
    "code": "PROPOSAL_STATE_CONFLICT",
    "message": "The proposal changed. Refresh and try again.",
    "field_errors": [{"field":"version","code":"stale"}],
    "correlation_id": "opaque-id",
    "retryable": false
  }
}
```

HTTP	Use
400	Malformed request
401	Unauthenticated / invalid token
403	Authenticated but prohibited
404	Absent or concealed resource
409	State/version/idempotency conflict
422	Semantically invalid command
429	Rate limit
500	Unexpected internal fault
503	Temporary dependency unavailable

9.2 Logging and observability

- Emit structured JSON logs with timestamp, level, service, environment, correlation ID, actor ID reference, route, latency, outcome, and safe error code.
- Propagate correlation IDs from Nginx through API, database spans, outbox, and workers.
- Metrics: request rate/error/latency, DB pool/saturation, queue depth/age, job failures, scan latency, object errors, login failures, submission/evaluation completion.
- Traces sample normal traffic and retain errors at higher rate; redact tokens, comments, file content, grades, and personal data.
- Alert on user-impacting SLO breach, queue age, backup failure, scan outage, repeated auth failures, and audit-integrity failure.

9.3 Pagination and filtering

Use cursor pagination for mutable high-volume lists and `page[size]` capped by configuration. Whitelist sort/filter fields; return `next_cursor`; normalize dates to ISO 8601; document timezone and inclusive/exclusive boundaries in OpenAPI.

SECTION 9 - CROSS-CUTTING CONCERNS

9.2 Caching, jobs, and file consistency

9.4 Caching strategy

Cache candidate	TTL / invalidation
Project-type catalogue	Minutes; invalidate on version activation
Permission resolution	Short; invalidate on assignment/user status change
Dashboard summaries	Short; refresh from outbox; detail remains authoritative
Signed file URLs	Very short; never cache beyond URL expiry
Rubric definitions	Version-keyed; immutable after approval
Not cached	Scores in draft, token secrets, permission decisions lacking scope

9.5 Asynchronous jobs

Queue	Examples	Reliability
critical	Outbox publish, malware result processing	Durable; highest priority; idempotent
notifications	Email/SMS/in-app dispatch	Retry with backoff; dedupe key; dead-letter
exports	PDF/CSV reports, archival packages	Per-user limits; progress; expiring output
imports	Users, enrollments, catalogue data	Validate first; error file; chunked commit policy
scheduled	Reminders, overdue checks, lifecycle cleanup	Distributed lock; recorded run; safe rerun

9.6 File storage and versioning

1. Create logical Document metadata and authorized upload intent.
2. Write bytes to quarantine using server-generated key; never trust original path.
3. Verify signature, checksum, size, scan result, and optional format-specific constraints.
4. Promote bytes to versioned object path; persist immutable DocumentVersion.
5. Issue authorized short-lived download; log access for protected classifications.
6. Lifecycle job deletes only under approved retention and hold checks.

9.7 Consistency and retries

All mutating APIs accept an idempotency key where user retry is likely. Use optimistic version columns on workflow aggregates. Workers store processed event IDs. External-provider timeouts produce UNKNOWN or RETRY state, never assumed success.

SECTION 10

10. Performance and scalability design

> **Target baseline.** The companion SRS targets p95 page response below 2 seconds and 500 concurrent users. These are acceptance targets to validate under an agreed workload model, not guarantees inferred from the stack.

10.1 Performance budget

Path	Design budget / tactic
Interactive API	p95 service response budget set below page target; avoid synchronous external calls
Page and list views	Code splitting, CDN, caching, indexed filters, cursor pagination, bounded results
Upload	Direct or streamed transfer; asynchronous scan; progress and resumability if approved
Reports	Asynchronous generation; status endpoint; expiring download
Dashboard	Precomputed summaries with visible freshness timestamp

10.2 Scaling design

- Keep app instances stateless; store shared state in PostgreSQL/Redis/object storage. Add instances behind Nginx based on CPU, latency, and queue indicators.
- Use connection pooling with explicit limits. Add a read replica for eligible reports only after measuring contention; never serve stale workflow decisions from it.
- Partition workers by queue and workload. Cap report/import concurrency so academic transactions retain resources.
- Serve hashed static assets from CDN; personalized responses default to private/no-store. Archive high-volume audit/event data only when retention and query requirements remain intact.

10.3 Index and query priorities

Query	Index direction
My active projects	session + membership/assignment + status
Upcoming deadlines	session + due_at + status
Evaluator workload	evaluator + stage + event status
Unread notifications	recipient + read_at + created_at
Pending outbox/jobs	status + available_at + priority
Audit search	occurred_at + actor/target/action, with partition plan if needed

10.4 Load-test release gate

Model 500 concurrent users across login, dashboards, uploads, proposals, scoring, and reports. Verify p95/p99 latency, errors, data-store saturation, queue age, and recovery after instance loss; also test an approved deadline burst.

APPENDIX A

Appendix A - Architecture Decision Records

An ADR is accepted only after review by the technology lead and affected owners. Until then, status is **Proposed**.

ADR	Decision / rationale	Status
ADR-001 Modular monolith first	One deployable backend with strict modules. Lowest operational cost; workflows need strong transactions. Extract only after measured need.	Proposed
ADR-002 Backend framework	Choose NestJS for one-language stack or Django REST Framework for mature Python ecosystem. Team capability is decisive.	Proposed
ADR-003 JWT access + rotated refresh	Short-lived stateless access; server-controlled renewal/revocation. Review browser token storage and SSO roadmap.	Proposed
ADR-004 PostgreSQL system of record	Relational integrity and transaction support fit academic workflow and scoring. JSONB only for bounded extension metadata.	Proposed
ADR-005 S3-compatible file store	Keep bytes outside DB; private objects, immutable keys, signed access, lifecycle policy.	Proposed
ADR-006 Outbox + worker	Commit events with academic transactions; deliver notifications/read models asynchronously and idempotently.	Proposed

ADR acceptance checklist

- Context and alternatives documented; operational cost understood.
- Security/privacy review complete; failure and rollback modes tested.
- Migration implications assessed; observability and ownership assigned.
- Decision linked to code/repository documentation and superseded ADRs retained.

Recommended first decisions

1. Select and staff the backend framework.
2. Confirm institutional identity/SSO and MFA direction.
3. Ratify tenant/session model and organization hierarchy.
4. Approve object storage, malware scanning, and retention services.
5. Confirm evaluation workflow and release authority before schema freeze.

APPENDIX B

Appendix B - Open design issues

B.1 Academic and governance decisions

ID	Open issue	Owner
TBD-01	Approved evaluation stages, criterion weights, pass rules, rounding, moderation thresholds	Academic process owner
TBD-02	Proposal approval chain and whether HOD review varies by program/type	Academic process owner
TBD-03	Group-size limits, supervisor capacity, co-supervision, exception authority	FYP governance
TBD-04	Deadline grace periods, extension evidence, late-submission behavior	FYP governance
TBD-05	Whether 9 hours/week and 300 hours are advisory, gating, or assessment evidence	FYP governance
TBD-06	AI-policy interpretation, declaration fields, review and appeal procedure	Academic integrity owner

B.2 Technical and operational decisions

ID	Open issue	Owner
TBD-07	NestJS vs Django REST Framework; API hosting model	Technology lead
TBD-08	SSO provider, account source, MFA scope, token/cookie architecture	Security / IT
TBD-09	File type/size limits, malware scanner, retention and legal hold	Security / records
TBD-10	Email/SMS providers, mandatory channels, delivery quotas	IT operations
TBD-11	Backup RPO/RTO, availability SLO, maintenance window, incident ownership	IT operations
TBD-12	LMS/SIS/calendar integrations and source-of-truth rules	Enterprise systems
TBD-13	Data residency, privacy notices, export/delete workflow	Privacy / legal
TBD-14	Accessibility target and supported browsers/devices	Product owner
TBD-15	Metric definitions and permissible analytics audiences	Academic process owner

> **Release rule.** A TBD that affects authorization, grades, policy enforcement, retention, or integration ownership must be resolved and recorded before the affected feature reaches production.

APPENDIX C

Appendix C - Developer handoff and glossary

C.1 Implementation handoff checklist

Workstream	Required artifact before build / release
Product / academic	Approved SRS, resolved policy configuration, role authority matrix, acceptance scenarios
Architecture	Accepted ADRs, module boundaries, threat model, integration contracts
API	OpenAPI 3 contract, error codes, idempotency rules, authorization annotations
Database	Physical ERD, migrations, indexes/constraints, seed/reference strategy, backup/restore test
Frontend	Route map, accessible component patterns, state/error/empty/loading designs
QA	Requirement traceability, state-transition tests, role/scope matrix tests, load/security plan
DevOps	Environment/IAM design, CI/CD, secrets, observability, runbooks, rollback
Security	SAST/DAST/dependency plan, upload test suite, audit review, penetration test scope

C.2 Definition of done for a module

- Domain invariants and transition tests pass; authorization includes endpoint and resource scope.
- API contract, validation, errors, idempotency, and audit behavior are documented.
- Database migration is reversible or has an approved forward-only recovery plan.
- Accessibility, empty/error/loading states, and responsive behavior are verified.
- Logs, metrics, alerts, privacy classification, and operational ownership exist.
- Integration, security, performance-impact, and user-acceptance checks pass.

C.3 Glossary

Term	Definition
FYP	Final Year Project.
RSCI	Riphah School of Computing & Innovation.
RBAC	Role-Based Access Control, augmented by resource context.
SDD	Software Design Document.
SRS	Software Requirements Specification.
Rubric snapshot	Immutable copy of criteria and scoring rules used for an evaluation.
Outbox	Events committed with business data for reliable asynchronous handling.
Idempotency	Repeated identical request produces one logical effect.
PITR	Point-in-time database recovery.
RPO / RTO	Acceptable data-loss window / service-recovery target.

> **Design completion.** This SDD is implementable as a baseline once the proposed ADRs and relevant TBDs are approved. It intentionally does not invent institutional marks, limits, or authorities absent from the supplied framework.